

[Project Report]

Secure Reverse Site-to-Site VPN Implementation

Author: Jacek Kaplon

Date: February 24, 2026

Status: Validated & Production-Ready

Stack: WireGuard, Linux Networking, Netfilter/iptables, NAT Traversal.

1. Project Overview

This project demonstrates the implementation of a **Reverse Site-to-Site VPN** architecture. The primary objective is to securely expose a private TCP service (Port 3000) hosted on an embedded Linux device (Octagon) located behind a dynamic ISP network.

2. Problem Statement: The Dynamic IP Challenge

The core challenge was to provide stable access to the internal service despite the following obstacles:

- **Dynamic IP Address:** Even with a public IP, the ISP frequently changes the address, which instantly terminates all active TCP sessions.
- **DNS Propagation Latency:** Relying on DDNS (Dynamic DNS) introduces significant downtime (minutes to hours) while waiting for DNS records to update globally.
- **Router Limitations:** Consumer-grade routers often lack the stability and advanced firewall features required for persistent service exposure.

3. The Solution: Reverse VPN Architecture

To solve the "Dynamic IP" problem, the standard client-server model was inverted. Instead of the server waiting for the client, the **Octagon (Node)** initiates an outbound encrypted tunnel to a **Static VPS (Hub)**.

3.1 NAT Traversal & Roaming

By utilizing WireGuard's native **Roaming** capability combined with **PersistentKeepalive = 25**, the tunnel remains active regardless of IP changes at the Node's location. If the ISP changes the home IP, WireGuard transparently updates the endpoint on the VPS within seconds, preserving the internal TCP session state.

4. Implementation Details

4.1 Network Addressing

- **VPS (Hub/Gateway):** Static Public IP | VPN IP: 10.10.10.2
- **Octagon (Node):** Dynamic Public IP | VPN IP: 10.10.10.1

4.2 Configuration Files

VPS (Hub) - `/etc/wireguard/wg0.conf`

```
Ini, TOML
[Interface]
Address = 10.10.10.2/24
ListenPort = 51821
PrivateKey = <VPS_PRIVATE_KEY>

[Peer]
PublicKey = <OCTAGON_PUBLIC_KEY>
AllowedIPs = 10.10.10.1/32
```

Octagon (Node) - `/etc/wireguard/wg0.conf`

```
Ini, TOML
[Interface]
Address = 10.10.10.1/24
PrivateKey = <OCTAGON_PRIVATE_KEY>

[Peer]
PublicKey = <VPS_PUBLIC_KEY>
Endpoint = <VPS_STATIC_PUBLIC_IP>:51821
AllowedIPs = 10.10.10.0/24
PersistentKeepalive = 25
```

5. Gateway Security & Traffic Routing (iptables)

The VPS acts as a **Security Gateway**, mapping public traffic to the encrypted tunnel via Destination NAT (DNAT).

```
Bash
# Enable IP Forwarding
sysctl -w net.ipv4.ip_forward=1

# Infrastructure & Service Access
iptables -I INPUT -p udp --dport 51821 -j ACCEPT
iptables -I INPUT -p tcp --dport 3000 -j ACCEPT

# Port Forwarding (DNAT) to VPN Client
iptables -t nat -A PREROUTING -p tcp --dport 3000 -j DNAT --to-destination 10.10.10.1:3000
iptables -I FORWARD -p tcp -d 10.10.10.1 --dport 3000 -j ACCEPT

# IP Masquerading (Persistence)
iptables -t nat -A POSTROUTING -o ens1 -j MASQUERADE
```

6. Testing & Validation

The following diagnostic suite confirms the integrity of the architecture:

- **Layer 3 Connectivity:** `ping 10.10.10.1` verified low-latency tunnel stability.
- **TCP Handshake (Telnet):** `telnet <VPS_IP> 3000` confirmed successful end-to-end mapping.
- **Packet Inspection:** `tcpdump -i wg0 -n tcp port 3000` was used to analyze the **SYN/ACK** flags, proving the DNAT logic is correctly forwarding packets to the Node.
- **Dynamic IP Roaming Test (Passed):** Forced a WAN IP change on the ISP router. The WireGuard `endpoint` updated automatically, and active TCP sessions remained stable without manual intervention.

7. Conclusion

This project demonstrates that a **Reverse VPN** strategy is the most robust method for bypassing Dynamic IP and NAT restrictions. It provides an enterprise-grade service uptime (High Availability) on consumer-grade internet connections.